



FDE Lab

LDPC Encoder Design

Course: Principle of FPGA Architecture and Its Applications

Name: R. Guo(郭睿康)

Student ID: 22307130115

Department:

School of Integrated Circuits and Micro-Nano Electronics Innovation

Major: Microelectronics Science and Engineering

1 Project Introduction

1.1 Project Goal

In this FDE Lab, we will implement two digital systems based on FDE board and related tool chain:

1. Lab1: Name Display

In this part, we are required to display our name based on the TextLCD Component in Rabbit, which is a virtual component written in C++ for waveform visualization.

2. Lab2: LDPC Encoder

LDPC (Low Density Parity Code) is one kind of ECC who plays an important role in error detection and correction. Compared with conventional ECC like BCH code、Hamming code etc., binary based LDPC has high error correction capability close to the Shannon limit. The basic principle of ECC is adding redundant verification bits to the information bits for extra information that can be used to verificate or correct the information interfered by noise. The process of adding redundant bits to the original message is called *encode*, while the process of recovering the original codes from the received encoded codes is called *Decode*. In this lab, we try to design an highly efficient encoder hardware for LDPC encode based on a encode scheme called QC-LDPC. And we'd display the encoded result using the LED component in Rabbit.

Through these two labs, we are desired to be more familiar with the details in complex digital system development and master the use of some important software in FPGA development, like Design Compiler/Yosys, rabbit and UFDE. Also, we can have a deeper understanding of the compilation flow in FPGA development including compiling, packing, placing, routing and generating bitstream.

1.2 Project Tool

1.2.1 FDE Board

This lab is based on FDEboard. The structure of the board is illustrated in **fig.1**. The board consists of two main chip, i.e. FDP3P7 and SMIMS Engine, and some peripheral device. FDP3P7 is an SRAM-based FPGA chip which can be

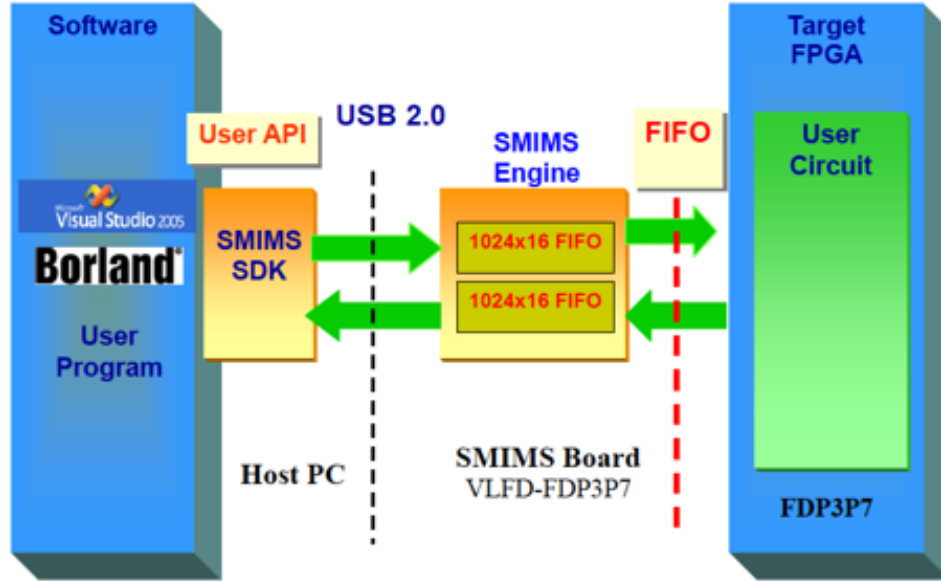


Fig. 1: Structure and work flow of FDEBoard

programed by the user, while the SMIMS Engine is a Flash-based chip provided by *Xilinx*, and this chip is for data transaction between FDEBoard and PC. Given the limited resources in this board, we can hardly efficiently display the lab consequence on board. We uses SMIMS Engine to transfer data between user computer and FDEBoard to display the output using the virtual component in user computer. The work flow of this process can be concluded as four steps:

- The computer sends a set of data through the SMIMS SDK into the input FIFO of the SMIMS Engine
- The data fed into the FIFO through the user circuit
- Processed data is sent to the output FIFO
- The computer reads the output FIFO data and uses virtual components to display

Due to the limitation of FIFO capacity, the system cannot work at a very high speed.

1.2.2 UFDE+ Design Flow

For FDE Lab, FDE designers have both provided optional DC Flow and Yosys Flow, which are illustrated in Figure 2, with either of them qualified to conduct the whole process.

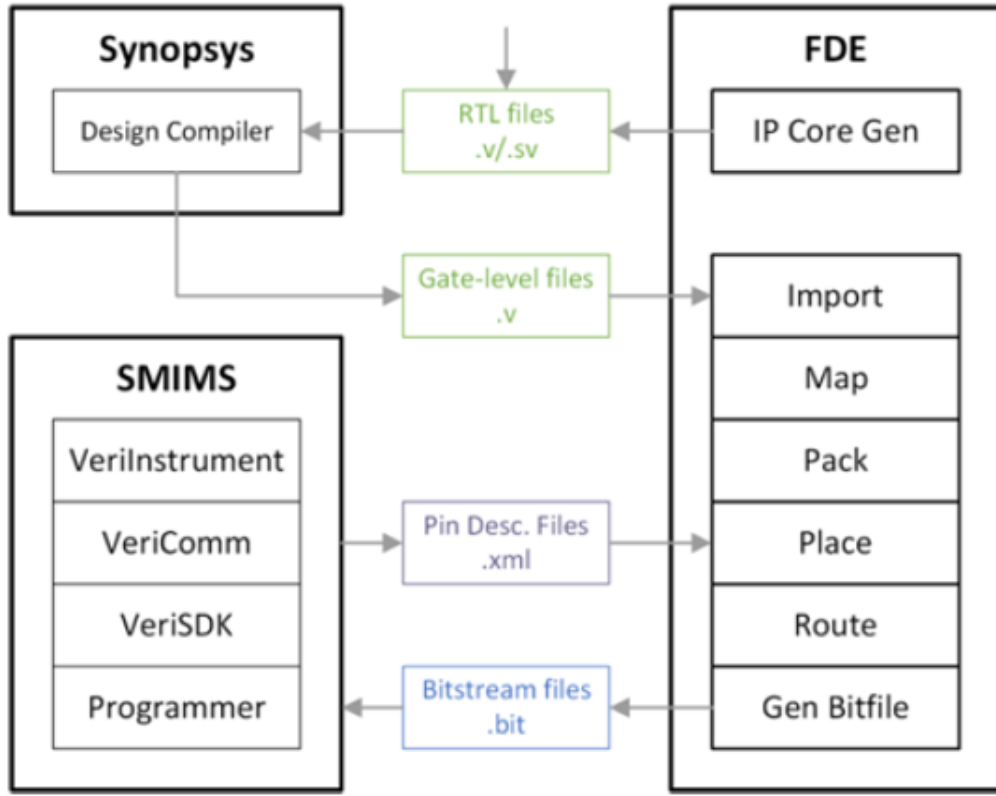


Fig. 2: FDE Dsign Flow illustration

Taking DC Flow in UFDE as an example, the development process of FDE experiments can be summarized as follows:

- Write RTL level Verilog files and verify the code through the behavior simulation in VCS or Vivado.
- Use Design Compiler to carry out logical synthesis of the code with relevant cell library and generate the gate-level netlist.
- Create a project in UFDE and import above netlist.
- Use UFDE to generate the pin constraint file automatically.
- Use UFDE to proceed to pack, place, route, and generate a bitstream file.
- Input the bitstream file and the pin constraint file into the Rabbit, add components and configure their ports and properties.
- Connect the FPGA board to the computer.
- Program the FPGA, set a working frequency and run.

2 Lab1: NameDisplay

2.1 TextLCD Introduction

The TextLCD component in Rabbit has two rows, each of which can display 16 characters. Its display starting at the top left corner, where the first character is shown. As each character is displayed, the display address automatically moves to the next position from upper left corner to bottom right corner. Once all 32 addresses (2 rows of 16 characters) are written, the character data in the initial position is overwritten in a cyclic manner.



Fig. 3: TextLCD Component

Ports of the Component and their functions are listed in the following table.

Tab. 1: Ports of TextLCD

Ports	Function
DB0~DB7	Input Data
RST	RESET Signal Clear all displayed data and move the cursor to the beginning position (top left).
EN	Transfer data at the falling edge of EN.

Additionally, the user manual provides the corresponding hexadecimal values for letters, symbols, and numbers, facilitating the design. It is odd that the codes have 0x20 offset compared with ASCII code.

➤ Character pattern

code	pattern	code	pattern	code	pattern	code	pattern	code	pattern
0x00	space	0x01	!	0x02	“	0x03	#	0x04	\$
0x05	%	0x06	&	0x07	‘	0x08	(	0x09	)
0x0A	*	0x0B	+	0x0C	,	0x0D	-	0x0E	.
0x0F	/	0x10	0	0x11	1	0x12	2	0x13	3
0x14	4	0x15	5	0x16	6	0x17	7	0x18	8
0x19	9	0x1A	:	0x1B	;	0x1C	<	0x1D	=
0x1E	>	0x1F	?	0x20	@	0x21	A	0x22	B
0x23	C	0x24	D	0x25	E	0x26	F	0x27	G
0x28	H	0x29	I	0x2A	J	0x2B	K	0x2C	L
0x2D	M	0x2E	N	0x2F	O	0x30	P	0x31	Q
0x32	R	0x33	S	0x34	T	0x35	U	0x36	V
0x37	W	0x38	X	0x39	Y	0x3A	Z	0x3B	[
0x3C	\	0x3D	]	0x3E	^	0x3F	_	0x40	`
0x41	a	0x42	b	0x43	c	0x44	d	0x45	e
0x46	f	0x47	g	0x48	h	0x49	i	0x4A	j
0x4B	k	0x4C	l	0x4D	m	0x4E	n	0x4F	o
0x50	p	0x51	q	0x52	r	0x53	s	0x54	t
0x55	u	0x56	v	0x57	w	0x58	x	0x59	y
0x5A	z	0x5B	{	0x5C		0x5D	}	0x5E	~
0x5F	0x5F ~ 0xFF space character								

Fig. 4: Characters Pattern

2.2 Lab Detials

My name is Ruikang Guo, so I want to show *** Welcome to ** Fudan University ++ Ruikang Guo ++* using TextLCD in two separate frames. This is not a hard task. I set a counter to match the address of *char_pos* of the component and current frame, and select hardcoded characters according to the *char_pos* and frame. Specifically, I set a 7-bit counter whose MSB indicates current frame while the LSB serves as EN of TextLCD.

You can find the v-src and tb file in FDELab\Namedisplay\RTL folder.

2.3 Lab Result

We use Vivado to perform behavior simulation. The simulation result of Vivado is demonstrated in **fig.5**

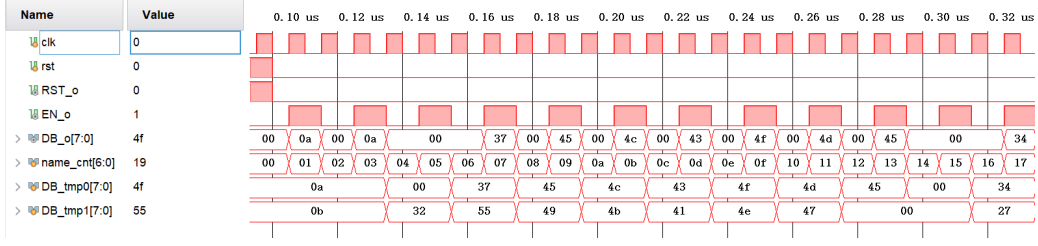


Fig. 5: Vivado Simulation Result

The experiment result in Rabbit is displayed in **fig.6**

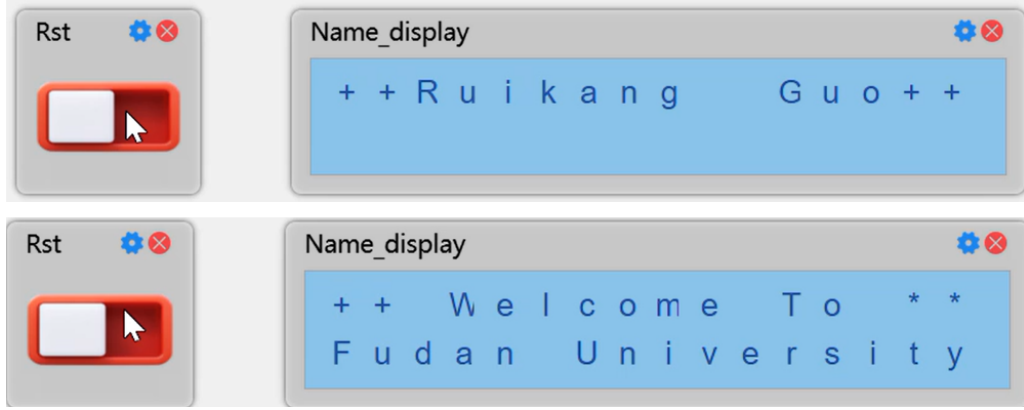


Fig. 6: Experiment Result

3 Lab2: LDPC Encoder

3.1 Brief Introduction of LDPC Encode

In this part, we are to introduce some basic theory concerning LDPC encode. LDPC is a kind of linear block code so the process of LDPC encode is actually a process of matrix multiply where the original message m is multiplied by a matrix G called *Generate Matrix*, and the result c is the encoded information with redundant bits added, i.e.

$$c = mG \quad (1)$$

As for binary LDPC, all elements in c, m, G is 0 or 1. Assuming that the length of m is k , and the length of c is n , then another $r = n - k$ bits of redundant information are added, and the scale of G is $k \times n$. This kind of LDPC is denoted as (n, k) LDPC, and the code rate is defined as $CR = k/n$. In practical use, n, k are rather big parameters scaling from 2^{13} to 2^{15} (e.g. (9216, 8192) LDPC is a kind of classical pattern). However, considering the limited resources in FDEBoard, we choose (96, 64) LDPC as a Demo, whose $CR = 2/3$.

According to the characteristics of Generate Matrix G , LDPC coding methods can be classified into many types. Among them, QC-LDPC is a hardware-friendly encoding method. In QC-LDPC, G is organized as follows:

$$G = \left[\begin{array}{cccc|ccccc} I & 0 & \cdots & 0 & G_{0,0} & G_{0,1} & G_{0,2} & \cdots & G_{0,q-1} \\ 0 & I & \cdots & 0 & G_{1,0} & G_{1,1} & G_{1,2} & \cdots & G_{1,q-1} \\ \vdots & \vdots & \ddots & \vdots & \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & I & G_{t-1,0} & G_{t-1,1} & G_{t-1,2} & \cdots & G_{t-1,q-1} \end{array} \right] \quad (2)$$

G in QC-LDPC consists of two parts. In the left there is an $k \times k$ Unit Matrix that is not necessary to store, while the right part of the matrix consists of sea of $G_{i,j}$. $G_{i,j}$ is a $s \times s$ circular shift matrix, whose first column can be right-shifted to generate the other columns of the matrix. It is easy to infer that $ts = k$, $qs = r$ and the encoded message c will be of the form of $c = [m|p]$, where m is the original message that is transfered while p is parity sequence. Each bit of the parity sequence can be depicted as

$$p_i = (m_0, m_1, \dots, m_{k-1}) \cdot (g_{0,i}, g_{1,i}, \dots, g_{k-1,i}) = \sum_{T=0}^{t-1} \sum_{S=0}^{s-1} m_S G_{T,L}(S, J) \quad (3)$$

where $i = s \times L + J$, and $G_{T,L}(S, J)$ is the (S, J) element of $G_{T,L}$. It is apparent that by using QC-LDPC, a large number of generator matrix parameters can be discarded and the encoding process can be performed in high parallelism.

3.2 System Architecture

The architecture of our encoder system refers to the paper *Research on Large Capacity Storage Circuit Technology and Implementation Circuits* Written by H. Zhang from Fudan University. The overall architecture of the system are shown below.

This encoder consists of seven main submodules.

- *Serdes* serves as an serial-to-parallel converter, collecting 4-cycle 4-bit input data before it delivers 16-bit data to the PU at a time.
- *Ctrl* is a system controller working together with the arbiter to distribute input data to counterpart PU.
- *PU* is the computing core where MAC happens, related detail will be discussed later.

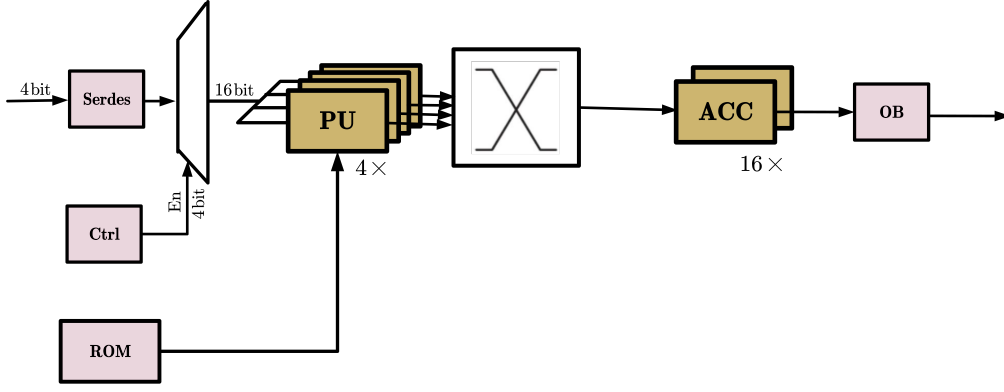


Fig. 7: System Architecture

- *ROM* is a Read-Only Memory where the first column of G_{ij} generated by software is stored
- *Crossbar* is a network that distribute the computing result of PU to correct accumulator.
- *ACC* is 16 global accumulators that accumulate data from four different PUs, each of which is responsible for one parity bit of one batch of Generate Matrix.

3.3 Module Details and Simulation

3.3.1 Parameters

It is necessary to introduce some important parameters used in the code here.

- **BATCH_DATA_WIDTH** is the width of input data;
- **BATCH_SIZE** means the number of Batches collected in *Serdes*.
- **FULL_DATA_WIDTH** is the width of *Serdes* and it is equal to the size of G_{ij} column length.
- **GENM_ROW** is the number of PUs.
- **GENM_COL** is the number of groups into which the matrix is divided.
- **DOUT_WIDTH** is the width of the output of the system.

3.3.2 Serdes

Serdes is a converter that turns serial data into parallel data which is important in reducing cycles for single data computation and increasing system throughput. I/O ports of *Serdes* is listed in the following figure.

```

module Serdes
#
(
    parameter BATCH_DATA_WIDTH = 4, // data width of each batch
    parameter BATCH_SIZE = 4, // number of batches
    parameter FULL_DATA_WIDTH = BATCH_DATA_WIDTH * BATCH_SIZE, // full data width
    parameter GENM_ROW = 4 // number of rows of generator matrix
)
(
    input wire clk, // clock
    input wire rst_n, // reset
    input wire[BATCH_DATA_WIDTH-1:0] din, // input data (one batch)
    input wire din_vld, // input data valid, high valid, indicate the input data is valid
    input wire [GENM_ROW-1:0] is_idle, // enable, high valid, indicate the system is idle
    output reg [FULL_DATA_WIDTH-1:0] dout, // output data (all batches concatenated)
    output reg dout_vld, // output data valid
    output wire din_ready // input data ready, high valid, indicate the system is ready to receive input data
);

```

Fig. 8: Serdes Ports

In our design, 4 bits are delivered to *Serdes* per valid cycle using an Valid-Ready handshake protocol, and every 4 valid cycles, 16 bits collected will be distributed to suitable PU according to the control signal generated by *Ctrl* Unit with *dout_vld* signal asserted. To implement this system, I instantiate an 4×4 -bit storage space to store data temporarily before they are emitted to the subsequent processes and set a pointer that always points at the position that is about to be filled. Once *din_vld* that indicates the input data is updated and valid, and *din_ready* that indicates the system is ready to receive data, are asserted simultaneously, input data will be latched in the position where the pointer sits, and the pointer will move to the next 4 bits that is empty. Specially, if the pointer is pointing at the highest 4 bits of the storage space when *din_vld* and *din_ready* are asserted, the pointer will move back to the lowest 4 bits and *is_full* signal will be asserted next cycle to drive the stored data to release and generate *dout_vld* that means the output data is updated. *is_idle* signal is given by the 4 PU to generate *din_ready* signal. This signal is set to prevent *Serdes* from collecting data if all the PUs are busy. You can find the *v_src* file in related folder and codes won't be demonstrated in the report.

3.3.3 Ctrl

Ctrl is the main control unit for the system that is responsible for data distribution. It is actually a counter that increments when new data is delivered which is reflected by asserting the *Serdes* data *vld* signal. Current value of the counter indicates where current data should be delivered to. *Ctrl*'s normal operation depends on the inherent sequential nature of data transmission. Ports of the

```

module Sys_Cnt
#
(
    parameter GENM_ROW = 4
)
(
    input wire clk,
    input wire rst_n,
    input wire d_vld,
    output reg [GENM_ROW-1:0] EN
);

```

Fig. 9: Ctrl Ports

module are listed in the following **fig.9**.

3.3.4 PU

PU(Processing Unit) is the most complex part of the system. Ports of this module is listed in the following **fig.10**, while its architecture is demonstrated in **fig.11**

In this module, there are some storage space (i.e. DinReg & GReg) where input data and Generate Matrix column are stored. MAC is operated on data in these space. MAC operation is actually reduced XOR of the AND results of 64 pairs of input data and Generate Matrix column. To shorten the critical path, 3 registers are inserted in the big XOR tree to construct pipeline. Once computation on one group of data completes, *Cnt/GCnt* that indicates the current column and group of Generate Matrix. If one column of matrix is computed, data in GReg will shift left when data in DinReg remains stable. If one group of matrix is computed, data in GReg will be updated but data in DinReg remains stable again until the whole matrix is computed when both of them are updated. All these data flows are driven by a FSM (which is not visible in the figure) controlled by Cnt, GCnt and some external signals.

Signal and FSM details will be discussed in this part. *din* is data collected by *Serdes* whose validity is indicated by *din_vld*. *EN* is a one-hot signal generated by *Ctrl* that activates the PU that is desired to receive the provided data.

```

module Processing_Unit
#
(
    parameter FULL_DATA_WIDTH = 16,
    parameter GENM_ROW = 4,
    parameter GENM_COL = 2,
    parameter IDLE = 2'b00,
    parameter CALC = 2'b01,
    parameter WAIT = 2'b10,
    parameter DONE = 2'b11
)
(
    input wire clk,
    input wire rst_n,
    input wire [FULL_DATA_WIDTH-1:0] din,
    input wire din_vld,
    input wire [FULL_DATA_WIDTH-1:0] rom_in_data,
    input wire EN,
    input wire all_done,
    output wire is_idle,
    output wire dout_vld,
    output wire [$clog2(FULL_DATA_WIDTH)-1:0] cnt_out,
    output wire dout_data,
    output wire is_done
);

```

Fig. 10: PU Ports

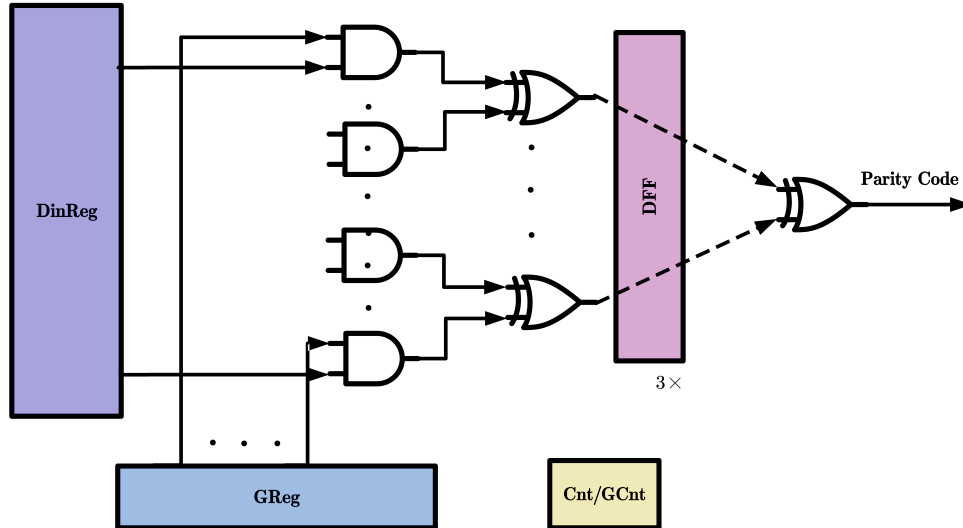


Fig. 11: PU Architecture

rom_in_data is Generate Matrix Column stored in ROM, and it will be received when one group of data is processed or when *Serdes* data is delivered to this PU. *cnt_out*, *dout_vld*, *dout_data* are signals related to output of PU. *dout_data* and *dout_vld* are output data and its valid signal, while *cnt_out* is control data

for subsequent CrossBar dispatch. Other signals are related to FSM and FSM of this PU are shown in **fig.12**.

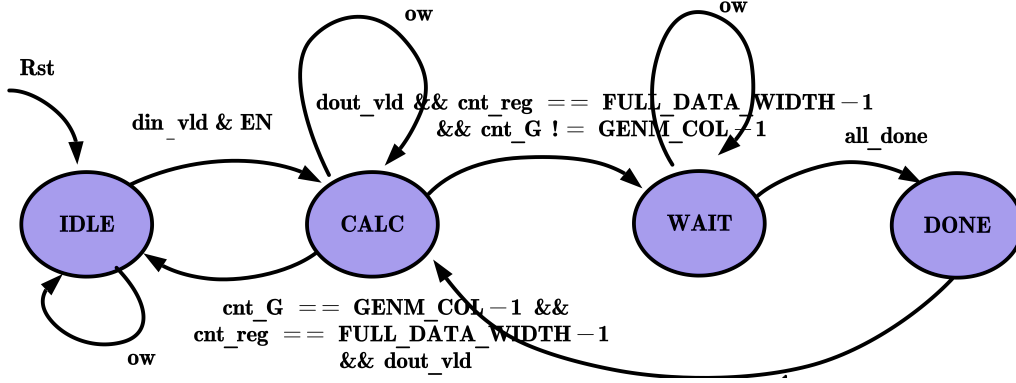


Fig. 12: PU FSM

As we can see in this graph, there are 4 states in this FSM

- **IDLE** is a default state where PU is reseted to. This state means the PU is ready to receive input data and matrix data.
- **CALC** means the PU is calculating parity codes right now.
- **WAIT** means the PU have completed one group of computation and wait for the other PU to complete their computation to avoid data from the same PU in different groups being accumulated in the same accumulator.
- **DONE** means all the PUs have completed one group of computation and ready to receive new matrix column.

The specific way of transferring between states can be seen in the code.

3.3.5 ROM

As depicted above, this ROM stores the **GENM_ROW** $\times$ **GENM_COL** first Columns of G_{ij} whose length is **FULL_DATA_WIDTH**. In this ROM, four pointers belong to 4 PUs are set pointing at data that will be receive by counterpart PU.

Fig.13 shows ports of ROM.

```

module ROM
#(
    parameter GENM_ROW = 4,
    parameter GENM_COL = 2,
    parameter FULL_DATA_WIDTH = 16
)
(
    input wire clk,
    input wire rst_n,
    input wire [GENM_ROW-1:0] is_idle,
    input wire [GENM_ROW-1:0] is_done,
    input wire [GENM_ROW-1:0] en,
    input wire dout_vld,
    output wire [FULL_DATA_WIDTH - 1:0] rom_out_data0,
    output wire [FULL_DATA_WIDTH - 1:0] rom_out_data1,
    output wire [FULL_DATA_WIDTH - 1:0] rom_out_data2,
    output wire [FULL_DATA_WIDTH - 1:0] rom_out_data3
);

```

Fig. 13: ROM Ports

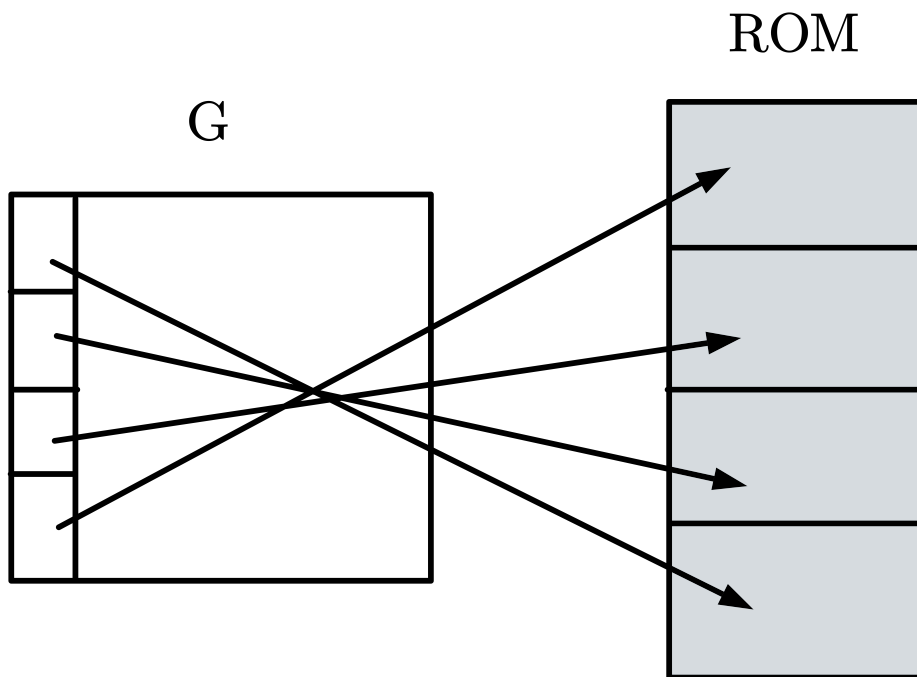


Fig. 14: Reverse Data Placement

It is necessary to note that considering the reverse order of matrix multiply, we should put matrix data in ROM reversely to make the answer correct.(As shown in **fig.14**)

3.3.6 Crossbar

This is a combinational *Crossbar* that dispatch data and its valid signal to correct accumulator.

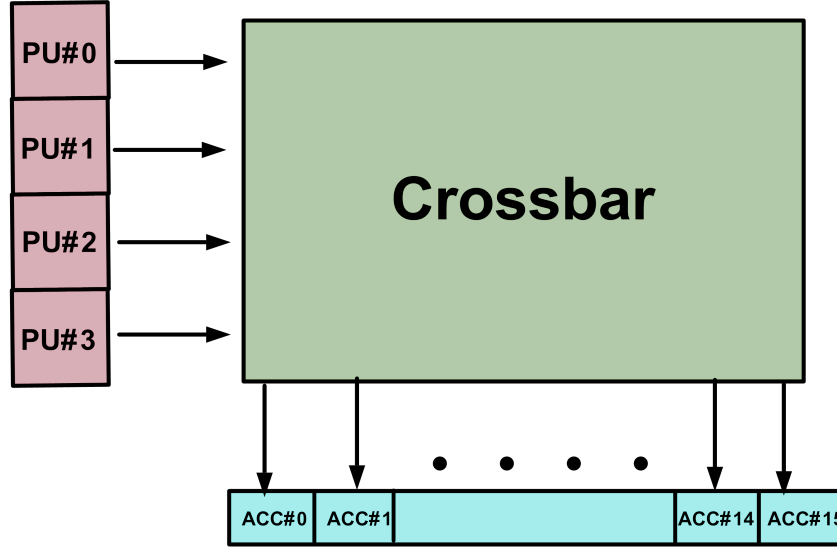


Fig. 15: Crossbar Arch

Considering the simultaneous arrival of data and the disordered nature of the individual operands in the MAC operation, we set the *acc_vld* signal to 3 bits to indicate how many PUs are currently pointing to the same ACC; and *acc_data* is set to be the sum of the outputs of these PUs. The following figure lists the ports of *Crossbar*.

```
module Crossbar#
(
    parameter GENM_ROW = 4,           // Number of rows in generator matrix
    parameter FULL_DATA_WIDTH = 16,   // Full data width
    parameter ACC_WIDTH = 16          // Width of accumulator data
)
(
    input wire [$clog2(FULL_DATA_WIDTH) * GENM_ROW -1:0] cnt_out,
    input wire [GENM_ROW-1:0] dout_int,
    input wire [GENM_ROW-1:0] dout_vld,
    output reg [47:0] acc_vld,
    output reg [ACC_WIDTH-1:0] acc_data
);
```

Fig. 16: Crossbar Ports

3.3.7 ACC

ACC is 16 global accumulators shared by 4 PUs. The result of different PUs' operations on the same column is accumulated in their corresponding accumulators. With the help of **WAIT** state to mask the data overwriting process of PUs to the accumulators, we can set a counter in ACC to count the number of PUs that have been counted, and increment the counter according to the value of *acc_vld*, and output the result to the output buffer when the value of the counter is increased to 4.

The following figure gives a list of *ACC*'s I/O ports.

```
module Accumulator #  
(  
    parameter GENM_ROW = 4  
)  
(  
    input wire clk,  
    input wire rst_n,  
    input wire [2:0] acc_vld,  
    input wire acc_data,  
    output wire acc_result,  
    output reg acc_result_vld  
);
```

Fig. 17: ACC Ports

3.3.8 Output Buffer

Since the computation between different PUs is completely independent (the progress of computation is determined by the speed of data input as well as the synchronization effect of the **WAIT** state), it is difficult to determine the generation time of a set of 16-bit outputs. Setting up a buffer at the output

side, and outputting the 16-bit data when it has been collected can effectively synchronize the output data. At the same time generate the corresponding output valid signal. Specifically in the output buffer set 16 data registers, and bit each data register to set a flag bit, when the data register to collect data, the flag bit will be asserted, to all the flag bit is asserted, the data will be output and generate valid signals, and the flag bit will be reset.

The following figure gives a list of *Output Buffer's* I/O ports.

```
module Output_Buffer #(
    parameter integer FULL_DATA_WIDTH = 16, // Number of accumulators, total bits to collect
    parameter integer DOUT_WIDTH = 16 // Parallel output width, should match FULL_DATA_WIDTH to output all bits
)(
    input wire clk,
    input wire rst_n, // Asynchronous active low reset
    input wire [FULL_DATA_WIDTH-1:0] acc_result, // Output from each accumulator (1 bit/accumulator)
    input wire [FULL_DATA_WIDTH-1:0] acc_result_vld, // Valid indicator for each accumulator output
    output reg [DOUT_WIDTH-1:0] dout, // Parallel output data
    output reg dout_vld, // Output valid pulse (1 clock cycle)
    output wire [FULL_DATA_WIDTH-1:0] led_filled
);
```

Fig. 18: OB Ports

It's important to point out that some of the ports in those modules above are set up for debugging and demonstration purposes, and have no real application utility.

3.3.9 Top module

In the top-level module, we set up a number of LEDs for displaying results and debug control signals, switches for data input and buttons for generating valid signals for data.

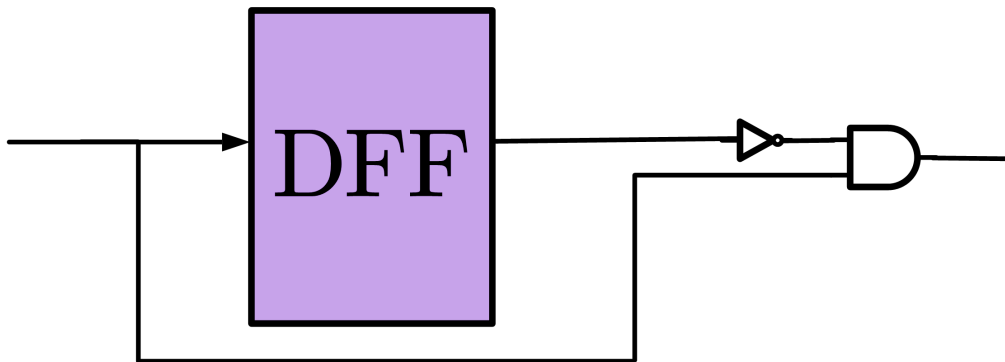


Fig. 19: LS2PUL Module

In the test, we found that when the clock cycle is relatively fast, the single data pressed out manually may be triggered by more than one clock edge causing the same data to be wrongly inputted repeatedly, so in the top module we also

set up a **long signal-to-pulse module(LS2PUL)**, which is realized with the help of a piece of combinational circuitry related to the original signal and the signal delayed by one cycle, which is in fact a DFF and an AND gate.

3.4 Simulation and Physical Test

We use Vivado 2019.1 to run behavior simulation and post-synthesis simulation on this device. You can find LDPC_encoder_tb.sv in RTL folder. Part of the simulation result waveform are in **fig.20** and **fig.21**.

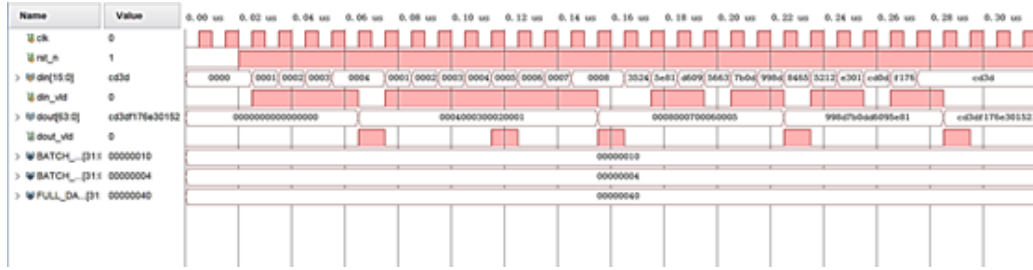


Fig. 20: Simulation for Serdes

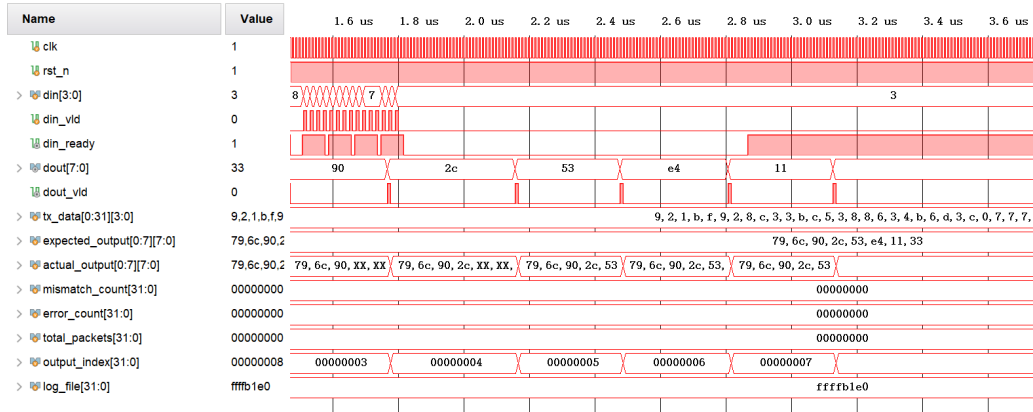


Fig. 21: Simulation for the whole system

You can also find simulation log in Demo folder where actual result and expected result generated by software are compared.

```

[Result Comparison]
Output[0] matches: fc
Output[1] matches: ee
Output[2] matches: 56
Output[3] matches: 72
Output[4] matches: 2e
Output[5] matches: 9d
Output[6] matches: 46
Output[7] matches: 72

[Test Statistics]
Total packets processed : 0
Total input errors      : 0
Total output mismatches : 0

=== Simulation Completed ===
End Time:                3135000

```

Fig. 22: Part of the simulation log

We use FDEBoard and Rabbit for physical verification and Demo. The layout of the component in Rabbit is as follows:

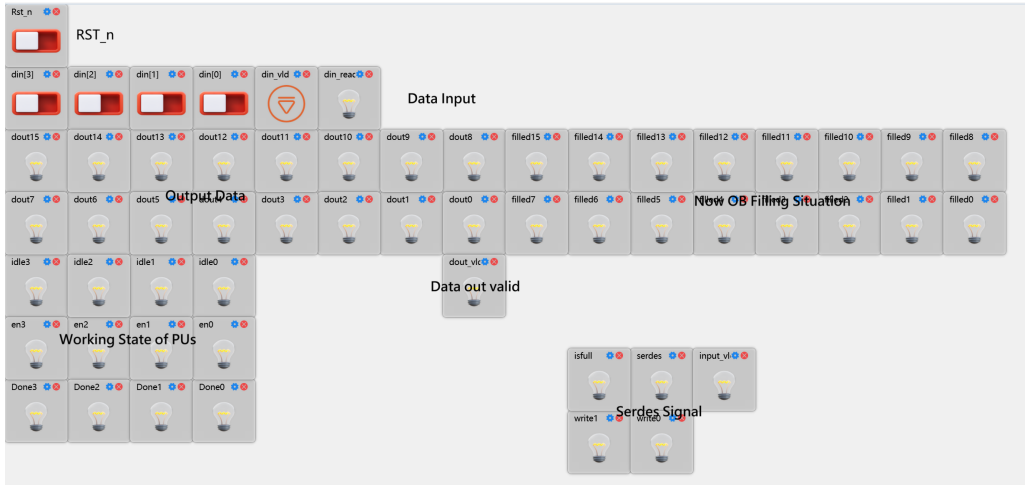


Fig. 23: Rabbit Layout

A Snapshot of the system's working status is shown in **fig.24**.



Fig. 24: Demo

4 Problems and Summary

4.1 Problems

4.1.1 Inconsistency Between the Simulation and on-board Result

Very often, the result is correct in Vivado simulation, while the result on board does not satisfy my expectation, even DC or Yosys process can not run properly. The reason is that Vivado can optimize some deficient logic in the design, while DC or Yosys have strict restrictions towards the code. Therefore, it is necessary to develop good coding standards when writing hardware code, such as reducing the use of logic such as division and the for loop.

4.1.2 DC's Failure to Find Related File

For many times, Design Compile can not find specific file when implementing synthesis. This is mainly due to two reasons, one is that there is a possibility that the code in the relevant file is written incorrectly, and the other is that new code has been added or modified but the DC's cache has not been cleared, which can be avoided by restarting the DC. You can avoid this problem by restarting the DC. Note that Rabbit has a similar problem.

4.1.3 Wrong Result in DFF

In the course of the experiment, we found that the results of DFF are very often different from the expected results, especially when the Q side needs to

change according to the value of the D-side data before the data jump at the clock edge where the D-side data jump occurs. I think there may have been a hold time violation. This can be well solved by increasing the optimization level during DC synthesis in the tcl file.

4.2 Summary

In this FDELab experiment, I did two main designs. One is the Name Display experiment and the other is the LDPC coding experiment.

The first design is very useful as it helps me to be familiar with the basic process of FDE lab. By completing this experiment, I practiced the development and boarding process of FDE, thus gaining a deep understanding, which will be of great help to the subsequent experiments.

And the second design was inspired by previous software simulations of LDPC in a lab on a related topic. the QC-LDPC coding is inherently hardware-friendly, which gave me a very strong desire to design a hardware-accelerated system to take advantage of the computational parallelism. In fact, at first I wanted to implement both an encoder and a decoder, but in practice I realized that the encoder already had a relatively high complexity and I did not implement the decoder for time reasons.

In summary, the FDE Lab provided a profound understanding of top-down design development, debugging, and the intricacies of FPGA-based system design. The hands-on experience of translating theoretical concepts into functioning digital systems illuminated the pivotal balance between hardware resource management and software control, reflecting the dynamic and problem-solving nature of engineering in the field of digital design.